# COMP101
# Introduction to Programming
# 2025-26

# Lecture-21
# global and local vars



1

# Scope of global and local vars

- A variable has a ‘global’ or a ‘local’ scope’
  - Determined automatically by where it is declared
- Global vars are declared outside a function
  - Can be used anywhere in the program
- Local vars are declared inside a function
  - only used by function in which they are declared
- Avoid over-using global vars (side-effects)

# Local variables

```
def double (num):                #parameter num = 9
    doubleNum = num * 2          #doubleNum is 'local var' and doubles the parameter
    print ("local var doublenum =", doubleocalNum)     #doubleNum = 18
    return                       #no return of values

def main():
    #explicit declaration no user i/p – not best idea
    num = 9     #don't do this or i/p in def main()
    double (num) #num is the argument to the function
    print ("num after the call =", num) #num=9

main()
```

A copy of num=9 is passed to the function
In the function, the copy is being used as parameter num.
Changes in the function only affect the copy (num) not the original which is outside of the function.
var num in def main() is not changed.

def main() calls def double()
with argument 9 passed to it;

def double doubles parameter **num** from 9 to 18 and function ends;
When function ends, control returns to next statement after the caller in def main();

In def double, **doubleNum** = 18
But **doubleNum** is 'local' to the function.
It's scope does not exist outside the function.

In def main before the call, num = 9;
After the call, num is still = 9

# Global variables

```
varForAll = 100          #global var = 100:  Can be used anywhere in the program
                         #declare outside of the functions and usually at the top
def funct1():            #no parameters
    funct1var = 5        #local var = 5:  Can only be used in the function
    print ("Value of local var in funct1 =", funct1var)     #local var =5
    print ("Value of global var in funct1 =', varForAll)    #global var =100
    return

def main():
    funct1()             #call without argument
    print ("Value of global var in main = ', varForAll)     #global var = 100

main()
```

If we attempt to use funct1var outside its function, we get an error

# Global variables - Problems

```
varForAll  = 100

def funct1():
    varForAll = 12
     print (“Value of local var in funct1 =",varForAll)        #o/p=12
    print (“Value of global var in funct1 =‘, varForAll)       #o/p=12 – access is to local var
    return

def main():
    funct1()
    print ("This is value of  global var,” varForAll)         #o/p=100 – access is to global var

main()
```

Don’t use the same global/local var names

varForAll is a ‘global variable’;
varForAll is a ‘local variable’ in the function but uses the same name as the global var and with a different value;
The local value 12 will be used in the function, not the global value of 100 as in the global var.
A local var can over-write a global var in the function

# Global variables - Problems

```
varForAll  = 100
```

Don’t use the same global/local var names

```
def funct1():
    global varForAll
    print (“Value of global var in funct1 =‘, varForAll)        #o/p=100 – access is to global var
    return

def main():
    funct1()
    print ("This is value of  global var,” varForAll)          #o/p=100 – access is to global var

main()
```

In the function, if we want access to the global var and its value of 100, put the word global in front of it;
Don’t assign it to anything or we change the original value;
We can now use the value=100 for it inside the function

# Global variables - Problems

```
varForAll  = 100

def funct1():
    global varForAll
     print (varForAll)       #o/p=100
     varForAll = 55
    print (varForAll)       #o/p = 55: global var has changed
    return

def main():
    funct1()
    print ("This is value of  global var," varForAll)          #o/p=55 – global var has been changed

main()
```

Don't use the same global/local var names:

We have used the statement global in the function to give us access to global variable varForAll so we can access the value 100;
PROBLEM: We have assigned local var with the same name:
varForAll = 55
This alters the value of the global var from 100 to a new value of 55